The Perceptron and Its Mistake Bound

It is guaranteed to stop. Does stopping mean it has learned anything?

32513 / 31005 Advanced Data Analytics Algorithms (Machine Learning), Spring 2026

Assignment A2, Option 1: study and implement a fundamental machine learning model.

Author: Hyunbeen Suk, 14655765

---

0. Link to implementation

```
https://colab.research.google.com/drive/1g66DwR6UeVGxuG4mS8Zsr81XSEF4AslW
```

The notebook `perceptron_mistake_bound.ipynb` is self-contained. Its first cell sets up the environment (NumPy, pandas and matplotlib are already on Colab and are installed if missing), and it generates all of its own data. No machine-learning library is used. "Notebook §3" means section 3 of the notebook, which is numbered separately from this report. The figures of code are taken directly from the notebook; their line numbers are the line numbers inside each notebook cell (in Colab: Tools → Settings → Editor → Show line numbers), ⋮ marks lines that were left out, and the highlighted lines are the ones the text refers to.

---

1. Problem definition

1.1 Interfaces

The task is binary linear classification. Because the object of study is a learning algorithm, there are two interfaces: one for training and one for using the trained classifier.

Training.

- Input: labelled examples $(x_1, y_1), \dots, (x_n, y_n)$ with $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$, a learning rate $\eta > 0$ and a maximum number of epochs.
- Output: a weight vector w (one hypothesis out of $\mathcal{H}$), and the number of mistakes made during training. My implementation also keeps two other weight vectors, the pocket and the average, which are used in §8.

Novikoff's theorem is about the second output. The research question asks whether it also says something about the first.

Using the classifier.

- Input: a point $x \in \mathbb{R}^d$.
- Output: a label (`predict`, Figure 2),

$$\hat{y} = \operatorname{sign}\left(w^\top x + b\right) \in \{-1, +1\}$$

The score $w^\top x + b$ is computed on the way, but it is not a probability. Its size depends on $\|w\|$, which depends on how many updates happened, so only its sign is used.

The data. All data is synthetic and generated in the notebook:

x	uniform on the unit sphere in $\mathbb{R}^d$ outside the band removed below, so $\|x\| = 1$; $d = 5$ ($d = 500$ in the coin-flip check of §6.2)
y	sign $\left((w^\star)^\top x\right)$, i.e. which side of a hidden unit vector $w^\star$ the point is on (random ± 1 in the coin-flip check)
margin	points with $\|(w^\star)^\top x\| < \gamma$ are thrown away, so the margin is γ (nothing is thrown away for the coin-flip labels)
noise	none in §6.1–6.2; in §6.3 and §8 each label is flipped with probability 0.06, so no separating hyperplane exists
sizes	training sets of 30 to 3,000 points; 2,000 each for validation and test (§8); 200,000 new points to measure the true error (5,000 in the coin-flip check)
classes	$+1$ and -1 are equally likely, because the sphere is symmetric

I generated the data instead of downloading a dataset for two reasons. First, the theorem is about $w^\star$ and γ, and I can only record the quantities in its proof if I know both (§4.3). Second, "has it learned?" is a question about the true error, the error rate on the whole distribution, and I can only measure that if I can keep drawing new points from the same distribution. The band around the boundary is removed using $w^\star$ only, before any model is trained. It is the assumption of the theorem, built in so the theorem can be tested. The coin-flip and label-noise experiments are the cases where the assumption does not hold. The cost is that nothing here is evidence about a real dataset (§9.4).

The generator is short (Figure 1).

notebook §2 Data and the bound · In [4]

```
1  def make_separable(n, d, gamma, seed=0, w_star=None):
2      """Passing an existing w_star draws more points from the same distribution."""
3      rng = np.random.default_rng(seed)
4      if w_star is None:
5          w_star = rng.normal(size=d)
6          w_star /= np.linalg.norm(w_star)
7      X = []
8      while len(X) < n:
9          # Gaussian points divided by their length are uniform on the sphere
10         b = rng.normal(size=(4 * n, d))
11         b /= np.linalg.norm(b, axis=1, keepdims=True)
12         # keep only points at least gamma away from the boundary
13         X.extend(b[np.abs(b @ w_star) >= gamma])
14     X = np.array(X[:n])
15     y = np.where(X @ w_star > 0, 1, -1)
16     return X, y, w_star
```

Figure 1. The data generator, `make_separable`.

Lines 10–11 draw Gaussian points and divide each by its length. A Gaussian looks the same in every direction, so the result is uniform on the unit sphere and $R = 1$ (before the bias coordinate of §3.2 is added). Line 13 keeps a point only if $|x^\top w^\star| \geq \gamma$; this is where the margin comes from, and it only looks at $w^\star$. Line 15 gives each point its label. If an existing $w^\star$ is passed in, lines 5–6 are skipped, which is how validation, test and evaluation points come from the same distribution as the training set.

1.2 What the model is

A single hyperplane. Points on one side are classified $+1$ and points on the other side -1. There is no hidden layer and no kernel. Because the model is this simple, there is an exact theorem about how it trains.

1.3 Why this is a learning problem

The labels come from a hyperplane, but the algorithm never sees that hyperplane, only labelled points. A non-learning solution would write the rule down, and that is exactly what is not possible here. This is the usual

situation in classification: a rule exists, nobody can write it down, and examples are available. Here all three hold in a controlled way. The rule exists with margin γ, it is hidden from the algorithm ($w^\star$ is only used to make labels and to score results), and I can draw as many examples as I want. What is not automatic is that future points come from the same distribution as the training points. That turns out to be one of the two conditions the answer depends on (§4.4).

---

2. Motivation

The perceptron (Rosenblatt, 1958) is one of the oldest trainable classifiers, and it comes with an exact guarantee about its own training:

> **Novikoff (1962).** If the data can be separated by a hyperplane with margin γ, and every point has $\|x_i\| \le R$, then the perceptron makes at most $(R/\gamma)^2$ mistakes before it stops. This does not depend on the number of examples or on the dimension.

I chose it for three reasons.

It can be checked. Most statements about models I have seen are qualitative. This one is a number: I can compute it before training, count the mistakes afterwards and compare.

The proof is short enough to run. The proof uses two inequalities about quantities I can record during training. So instead of only quoting the theorem, I record those quantities after every mistake and check the inequalities (§6.1).

Its history connects to every part of the project. Rosenblatt (1958) proposed the perceptron: a single layer of adjustable weights that learns to classify from examples. Novikoff (1962) proved that on separable data its update rule stops after at most $(R/\gamma)^2$ mistakes, which is the theorem studied in §4. Minsky and Papert (1969) then showed what a single perceptron cannot represent, with XOR as the best-known example. That is a limit of the hypothesis space, not of the training rule (§3.1), and research on perceptrons slowed down for many years after the book. Later work dealt with the two questions the original theorem leaves open: what to do when the data cannot be separated, and what happens on new data. Gallant (1990) proposed the pocket algorithm, which keeps the best weights seen so far (§8). Freund and Schapire (1999) brought the perceptron back with the voted and averaged versions (I use the averaged one in §8), and used a leave-one-out argument to turn the mistake bound into a statement about unseen data, which is the step this project depends on (§4.4).

2.1 Research question

> **The perceptron is guaranteed to stop. Does stopping mean it has learned anything?**

Stopping is about the training data: Novikoff's theorem says that on separable data the perceptron eventually gets every training example right. Learning is about new data: the error rate on the whole distribution, which I call the true error. The theorem only talks about the first. It holds for any order of the examples, even one chosen to be as unhelpful as possible, so on its own it cannot say anything about new points. The intuitive answer, that a perceptron that has stopped must have learned something, turns out to be false (§6.2).

So the question becomes: if stopping is not enough, what does guarantee learning, and does that hold up when measured? §6 answers this in three steps:

	What it shows
§6.1	The theorem holds, and both inequalities of its proof can be seen during training
§6.2	Stopping alone is not learning (coin-flip labels). With i.i.d. data from a distribution that has a margin, the number of mistakes per example bounds the expected true error, and the measurements agree
§6.3	With label noise the guarantee no longer applies, and the textbook algorithm has no defined result

§7 compares the three guarantees used in the project, and §8 tests two fixes for the problem found in §6.3.

2.2 How the question was reached

The perceptron and its update rule are taught in the subject (module 2, pages 2.1–2.5, and the linear-model tutorial), and Hoeffding's bound is on page 3.9. Novikoff's theorem is not in the pages I worked from. The AI assistant suggested it while I was choosing a model, as a result that would let me test a claim about the algorithm instead of just quoting it. The statement and proof are written out in §4 and checked by the notebook (§5, §6.1). The leave-one-out step in §4.4 comes from Freund and Schapire (1999).

My first plan was to check the theorem itself: does the bound hold, how tight is it, and what happens when the data cannot be separated. These only check a known result, so I turned them into one question about the model itself: it is guaranteed to stop, but does stopping mean it has learned? My first answer ("no, because the bound ignores n") was wrong, and correcting it gave the answer in this report (§10.2, challenge 4). The checks of the first plan are still here, as §6.1 and §6.3.

---

3. Machine learning approach

Overview. The notebook works in five steps:

1. **Data.** `make_separable` draws points with a known margin, and `draw` adds label noise (notebook §2 and §5).
2. **Bias.** `add_bias` appends the constant 1 to every input (§3.2).
3. **Training.** `Perceptron.fit` runs the update rule. It returns three weight vectors (the last one, the pocket and the average), the number of mistakes and, if $w^\star$ is given, the two quantities of the proof (§3.4, §4.3, §8.1).
4. **Evaluation.** `accuracy` is used on validation data to choose, on test data once, and on 200,000 new points (5,000 in the coin-flip check) to measure the true error. Hoeffding's bound gives the test interval (§3.6, §7).
5. **Use.** `predict` gives the label of a new point (§3.1).

Everything is written in NumPy, because Option 1 asks for the algorithm itself to be implemented; pandas and matplotlib are only used for tables and plots.

3.1 Hypothesis space $\mathcal{H}$

$$\mathcal{H} = \left\{ x \mapsto \operatorname{sign}(w^\top x + b) \ : \ w \in \mathbb{R}^d,\ b \in \mathbb{R} \right\}$$

Each hypothesis is a hyperplane: w is the normal vector (the direction perpendicular to the plane) and b moves the plane away from the origin.

In the notebook a hypothesis is just its weight vector (Figure 2).

notebook §1 Model · In [3]

```
61 def predict(w, X):
62     return np.where(Perceptron.add_bias(X) @ w >= 0, 1, -1)
63
64 def accuracy(w, X, y):
65     return np.mean(predict(w, X) == y)
```

Figure 2. Prediction and accuracy for a given weight vector.

Line 62 computes $w^\top x + b$ for all rows of `X` at once and takes the sign. A score of exactly 0 is sent to $+1$; with continuous data this practically never happens. Both functions take `w` as an argument because one training run returns three different weight vectors (§8.1), and all three are scored with the same code.

Two properties of $\mathcal{H}$ are used later.

Scaling w does not change the hypothesis. (w, b) and (cw, cb) with $c > 0$ give the same sign for every input, so they are the same classifier. §3.5 uses this to show that the learning rate does not matter.

XOR is not in $\mathcal{H}$. No single line separates $\{(0,0),(1,1)\}$ from $\{(0,1),(1,0)\}$. This is a limit of the hypothesis space. No training procedure, learning rate or amount of data can fix it.

3.2 The bias as an extra coordinate

I append a constant 1 to every input, $x \to [x_1, \ldots, x_d, 1]$, and put b at the end of w. Then $w^\top x + b$ becomes a single dot product, and the code and the proof only need one form. From here on, x and w include this extra coordinate.

This changes the radius. Every point now has $\|x\|^2 = 1 + 1 = 2$, so $R^2 = 2$ instead of 1. A bound computed with the old R would be half the correct size for the algorithm that actually runs, so `bound_inputs` measures R after adding the 1 (Figure 3; §10.2, challenge 2).

notebook §1 Model · In [3]

```
6     @staticmethod
7     def add_bias(X):
8         X = np.asarray(X, float)
9         return np.hstack([X, np.ones((len(X), 1))])
```

notebook §2 Data and the bound · In [4]

```
18 def bound_inputs(X, y, w_star):
19     Xa = Perceptron.add_bias(X)
20     R = np.max(np.linalg.norm(Xa, axis=1))
21     # the true boundary goes through the origin, so its bias weight is 0
22     w_star_a = np.append(w_star, 0.0)
23     gamma = np.min(y * (Xa @ w_star_a))
24     return R, gamma, w_star_a
```

Figure 3. Adding the bias coordinate, and measuring R and γ after adding it.

Line 9 appends the column of ones. Training, prediction and the loss all call this one function, so they always agree on the extra coordinate. In `bound_inputs`, line 20 measures R on the extended points. Line 22 adds a 0 to $w^\star$, because the true boundary goes through the origin; this also keeps $\|w^\star\| = 1$, which §4.1 needs. Line 23 measures γ as the smallest $y_i\,(w^\star)^\top x_i$ in the sample.

3.3 Loss L

The perceptron is usually presented as an update rule, but it also has a loss, the perceptron criterion:

$$L(w) = \frac{1}{n}\sum_{i=1}^{n} \max\left(0,\ -y_i\, w^\top x_i\right)$$

A correctly classified point adds 0. A misclassified point adds $-y_i\, w^\top x_i$, which is larger the further the point is on the wrong side. For one misclassified point the gradient is $-y_i x_i$, so a stochastic gradient step

$$w \leftarrow w - \eta\, \nabla L_i = w + \eta\, y_i x_i$$

is exactly the perceptron update. (Strictly it is a subgradient, because $\max(0, \cdot)$ has a corner at 0, but on a misclassified point the formula is the same.)

notebook §1 Model · In [3]

```
58 def perceptron_loss(w, X, y):
59     return np.mean(np.maximum(0, -y * (Perceptron.add_bias(X) @ w)))
```

Figure 4. The perceptron loss.

Line 59 is the formula: `-y * (Perceptron.add_bias(X) @ w)` is $-y_i\, w^\top x_i$ for all points, `np.maximum(0, ...)` is the max, and `np.mean` is the average. Training never calls this function; it only takes the update step (line 32 of Figure 5). I compute the loss only afterwards, in §8.2.

Two things about this loss were not in the lecture notes, and both matter later.

It is a substitute for the real objective. What we care about is the 0/1 error, but it is flat almost everywhere, so its gradient gives no direction. L is a convex stand-in that does have a useful gradient.

It has a useless minimum. $w = 0$ gives $L(0) = 0$, the smallest possible value, but classifies nothing. So the perceptron is not just "minimise L". It starts at $w = 0$ and updates when $y_i w^\top x_i \le 0$. Because the condition is $\le$ and not $<$, the very first example triggers an update and the algorithm leaves $w = 0$.

3.4 Algorithm A

$w \leftarrow 0$
repeat until a full pass makes no mistake:
  for each (x_i, y_i) :
    if $y_i\, w^\top x_i \le 0$: $\quad w \leftarrow w + \eta\, y_i x_i$

Why the update goes in the right direction. After updating on a misclassified x_i, the score of that point becomes

$$y_i(w + \eta y_i x_i)^\top x_i = y_i w^\top x_i + \eta\, y_i^2 \|x_i\|^2 = y_i w^\top x_i + \eta \|x_i\|^2$$

which is larger than before by $\eta\|x_i\|^2$. So the update always pushes the point towards the correct side. It may not fix it in one step, and it may break other points, which is why the proof in §4 needs two inequalities.

Each line of the algorithm above is one line of `Perceptron.fit` (Figure 5).

notebook §1 Model · In [3]

```
11      def fit(self, X, y, w_star=None):
        ⋮
13          Xa = self.add_bias(X)
14          y = np.asarray(y, int)
15          n, d = Xa.shape
16          assert set(np.unique(y)) <= {-1, 1}, 'labels must be -1 / +1'
17
18          # start at w = 0 (the proof needs this)
19          w = np.zeros(d)
        ⋮
26          for epoch in range(self.max_epochs):
27              errs = 0
28              # same order every epoch (the leave-one-out argument needs a fixed order)
29              for i in range(n):
30                  # a mistake is y_i * w.x_i <= 0; 0 counts, so the algorithm can leave w = 0
31                  if y[i] * (Xa[i] @ w) <= 0:
32                      w = w + self.lr * y[i] * Xa[i]
        ⋮
48              if errs == 0:
49                  self.converged = True
50                  break
```

Figure 5. The training loop of `Perceptron.fit`. The lines for §4.3 and §8 are left out here and shown in Figures 6 and 12.

algorithm	line	code
$w \leftarrow 0$	19	`w = np.zeros(d)`, the start the proof needs
repeat until a full pass makes no mistake	26, 48–50	stop after a pass with `errs == 0`; `max_epochs` limits the number of passes when that never happens
for each (x_i, y_i)	29	the same order in every pass (§3.5)
if $y_i\, w^\top x_i \le 0$	31	`<=`, not `<` (§3.3)
$w \leftarrow w + \eta\, y_i x_i$	32	the update

Line 13 adds the bias coordinate. Line 16 checks that the labels are -1 and $+1$: the update adds $y_i x_i$, so with labels 0 and 1 a point of class 0 would add nothing, count as a mistake on every pass, and training would never stop.

3.5 Training procedure and hyperparameters

	Setting	Reason
Initial weights	$w = 0$	needed by the proof, and it is how the algorithm leaves the useless minimum of §3.3
Learning rate η	1.0	it does not matter: starting from 0, every update adds $\eta\, y_i x_i$, so with rate η the weights are always exactly η times the weights with rate 1. Same signs, so the same mistakes and predictions (§3.1). This would not hold if w started somewhere else
Epoch limit	10,000 on separable data (never reached); 400 on noisy data	separable data stops by itself after at most $(R/\gamma)^2$ mistakes; noisy data never stops, so the limit ends training
Example order	fixed in every run	the bound holds for every order, and the leave-one-out argument in §4.4 needs a fixed order
Validation and test (§8)	new samples of 2,000 points each	the data is synthetic, so nothing has to be taken out of the training set

Nothing is tuned. The only choice made on validation data is which of the three versions in §8 to use. The inputs need no scaling because every x already has length 1. There is no regularisation and no learning-rate schedule; both would break the argument that the learning rate does not matter.

3.6 Evaluation metrics

- **Total mistakes**: what the theorem bounds. Divided by $n+1$ it becomes mistakes per example, which §4.4 connects to the true error and which the answer depends on (§6.2).
- **True error**: the error rate on the whole distribution, measured on 200,000 new points (5,000 in the coin-flip check). This is what "has it learned?" means. By Hoeffding's bound (module 3.9) each measurement is within ± 0.003 of the exact value with 95% confidence (§7).
- **Validation and test accuracy**: for the noisy experiments in §8. Validation is used to choose, and the test set is used once to report. The classes are balanced, so accuracy is a fair measure.
- **The ceiling**: the true accuracy of $w^\star$ itself. When each label is flipped with probability 0.06, no classifier can do better, so this is the right target instead of 100%.
- **Mean and standard deviation over 20 training sets**: to compare the versions in §8.2, because one training set is one draw of luck.
- **Perceptron loss**: reported next to $\|w\|$ in §8.2 to show what it actually measures (§9.2).

---

4. The theorem and its proof

4.1 Statement

Suppose there is a unit vector $w^\star$ ($\|w^\star\| = 1$) and a margin $\gamma > 0$ such that

$$y_i \, (w^\star)^\top x_i \;\geq\; \gamma \quad \text{and} \quad \|x_i\| \leq R \qquad \text{for every } i.$$

Then the perceptron, started at $w = 0$, makes at most $(R/\gamma)^2$ mistakes.

4.2 Proof

Let w_k be the weight vector after the k-th mistake, made on example (x_i, y_i). I use $\eta = 1$, which is allowed by §3.5.

(a) Each mistake moves w at least γ further along $w^\star$.

$$w_k^\top w^\star = (w_{k-1} + y_i x_i)^\top w^\star = w_{k-1}^\top w^\star + y_i \, x_i^\top w^\star \;\geq\; w_{k-1}^\top w^\star + \gamma$$

because $y_i \, x_i^\top w^\star \geq \gamma$ by the assumption. Starting from $w_0 = 0$ and repeating k times gives $w_k^\top w^\star \geq k\gamma$.

(b) Each mistake increases $\|w\|^2$ by at most R^2.

$$\|w_k\|^2 = \|w_{k-1} + y_i x_i\|^2 = \|w_{k-1}\|^2 + 2\, y_i \, w_{k-1}^\top x_i + \|x_i\|^2 \;\leq\; \|w_{k-1}\|^2 + R^2$$

The middle term is ≤ 0 because this was a mistake ($y_i w_{k-1}^\top x_i \leq 0$), and $\|x_i\|^2 \leq R^2$. Repeating k times gives $\|w_k\|^2 \leq kR^2$. This is the only place where the "only update on a mistake" rule is used; (a) would hold for any update.

(c) Put them together. Since $\|w^\star\| = 1$, the Cauchy–Schwarz inequality gives $w_k^\top w^\star \leq \|w_k\|$. So

$$k\gamma \;\leq\; w_k^\top w^\star \;\leq\; \|w_k\| \;\leq\; \sqrt{k}\, R \qquad \Longrightarrow \qquad k \leq (R/\gamma)^2 \qquad \blacksquare$$

Neither n nor d appears anywhere in the proof. It counts mistakes, not examples, and each mistake has to make a fixed amount of progress.

4.3 Recording the proof during training

Both inequalities are about quantities I can record during training: after every mistake, store $w_k^\top w^\star$ and $\|w_k\|^2$. The proof says the first can never go below the line $k\gamma$ and the second can never go above the line kR^2. `Perceptron.fit(..., w_star=...)` does this (Figure 6). It is only possible because I know $w^\star$, which is one of the reasons for using synthetic data (§1.1).

notebook §1 Model · In [3]

```
31                 if y[i] * (Xa[i] @ w) <= 0:
32                     w = w + self.lr * y[i] * Xa[i]
33                     errs += 1
34                     if w_star is not None:
35                         self.proj.append(w @ w_star)
36                         self.normsq.append(w @ w)
```

notebook §3 Checking the proof during training · In [5]

```
 1 proj, normsq = np.array(model.proj), np.array(model.normsq)
 2 k = np.arange(1, len(proj) + 1)
   ⋮
17 a_ok = np.all(proj >= k * gamma - 1e-9)       # tiny tolerance for rounding
18 b_ok = np.all(normsq <= k * R ** 2 + 1e-9)
19 print(f'(a) held at all {len(k)} mistakes: {a_ok}   (b) held at all {len(k)} mistakes: {b_ok}')
```

Figure 6. The two quantities of the proof, recorded inside `fit` and checked in notebook §3.

Lines 34–36 run only after an update, so `w` is already w_k; they store $w_k^\top w^\star$ and $\|w_k\|^2$. The $w^\star$ passed in is the one with the extra 0 from Figure 3. In notebook §3, line 2 numbers the mistakes $k = 1, 2, \ldots$, and lines 17–18 check (a) and (b) at every k, with R and γ from `bound_inputs`. The `1e-9` only allows for rounding error. Figure 7 in §6.1 plots the same values.

4.4 From the mistake bound to unseen data

Everything so far is about the training data. To say something about new points, I need to connect them to the training points. The usual way is to assume all points, training and future, are drawn independently from the same distribution (i.i.d.). Freund and Schapire (1999) use a leave-one-out argument to turn the mistake bound into a bound on the expected true error. They prove it for a different version of the algorithm (the voted perceptron) under weaker conditions. I use a simple version that matches my code: the last weight vector, a fixed order, training until a pass with no mistakes, and a distribution where every point has margin at least γ.

Draw $n + 1$ points, and let M be the number of mistakes when training on all of them. The argument has three steps.

1. **Leaving one point out is like testing on a new point.** Train on all points except point i, then test on point i. That classifier never saw point i, and point i comes from the same distribution, so on average this test error equals the true error of a perceptron trained on n points.
2. **Only points that caused a mistake can be wrong when left out.** If training never updated on point i, then removing it changes nothing: the same updates happen in the same order and the same w comes out. That w classifies point i correctly, because the last pass went over point i without a mistake. So at most M of the $n + 1$ left-out tests can be wrong.
3. **Novikoff limits M.** By §4.2, $M \le (R/\gamma)^2$.

Together:

$$\mathbb{E}[\text{true error after } n \text{ points}] \;\le\; \frac{\mathbb{E}[M]}{n+1} \;\le\; \frac{(R/\gamma)^2}{n+1}$$

Some remarks that matter for the research question:

- **It needs two conditions.** Step 1 needs i.i.d. data; this is the only place new data comes in. Step 3 needs the margin to hold for the whole distribution, not just the sample, so that every possible training set has at most $(R/\gamma)^2$ mistakes. Without that margin, steps 1 and 2 still give $\mathbb{E}[M]/(n+1)$, but M is not limited and the bound can be useless (§6.2, coin flips).
- **The bound falls because the limit on the mistakes does not grow.** The top, $(R/\gamma)^2$, is fixed by §4.2, and the bottom grows with n.
- **Its limits.** It is about the average over training sets, not about one particular training set. It needs a margin, so it disappears with label noise. And it needs a fixed order; with shuffling, removing one point changes the order of the others and step 2 no longer works. All my runs use a fixed order.

---

5. Checking the implementation

Good accuracy does not prove the code is correct, so I checked it against the theory instead. Each likely bug would show up in a specific place:

If the code had this bug	it would show here	What happened
update in the wrong direction	inequality (a) fails at the first mistake: from $w = 0$ a wrong-sign update gives $w_1^\top w^\star \le -\gamma$	(a) held at all 18 mistakes of the recorded run (Figure 6, lines 17–18; Figure 7)
wrong mistake condition or stopping rule	separable runs would not stop with every training point correct, or would make more than $(R/\gamma)^2$ mistakes	the recorded run stopped after 2 epochs with 18 mistakes (bound 193); every coin-flip run stopped with training error 0 (§6.2)

That the update is a gradient step on L is not tested separately, because it is the short derivation in §3.3.

---

6. Results

6.1 The bound holds, and the proof can be seen

On one run ($n = 200$, $d = 5$, measured margin $\gamma = 0.102$, $R = 1.414$) the bound is 193 mistakes and the perceptron made 18, stopping after 2 epochs. Both inequalities held at all 18 mistakes.

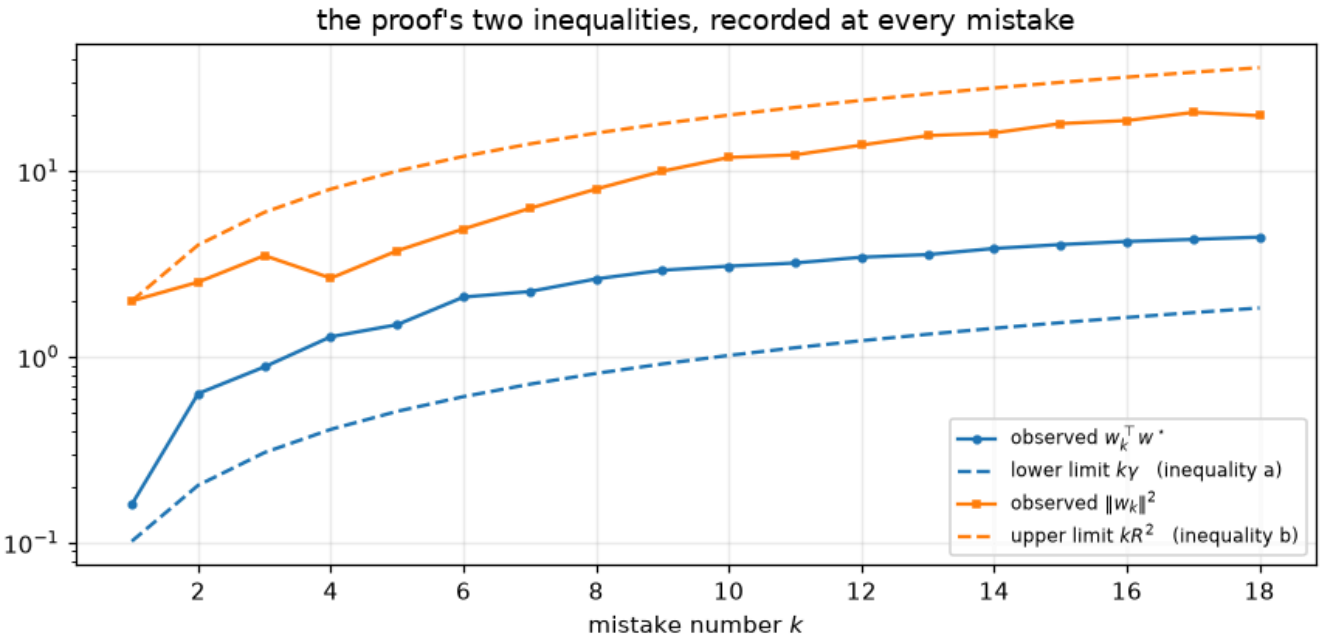


Figure 7. The two quantities of the proof after every mistake, with the limits from the proof (log scale): $w_k^\top w^\star$ never goes below $k\gamma$ (a) and $\|w_k\|^2$ never goes above kR^2 (b); at $k = 1$ the second touches its limit, because $\|w_1\|^2 = \|x_i\|^2 = R^2$. By §4.2(c) both cannot hold beyond $k = (R/\gamma)^2 = 193$; training stopped at 18.

The bound is about ten times larger than what happened. That is expected: it covers the worst possible order of examples, while my data comes in one ordinary order. The bound also does not grow with n or d, and §6.2 depends on that.

6.2 Does stopping mean learning?

First, stopping without learning. I labelled 200 points in 500 dimensions with coin flips, so there is no rule to learn. With more dimensions than points, any labelling can be separated, so the perceptron stops after memorising the sample. For comparison, the same sizes with labels from a real rule $w^\star$ with margin 0.10. Each model is scored on 5,000 new points, three runs each:

labels	stopped	training error	mistakes / $(n+1)$	true error (5,000 new points)
coin flip	3 of 3	0	1.13 – 1.42	0.492 – 0.497
real rule $w^\star$	3 of 3	0	0.17 – 0.20	0.015 – 0.026

All six runs stopped with every training point correct, but with coin flips the true error is about 0.5, which is guessing. This is not luck of three runs: coin-flip labels have nothing to do with x, so every classifier has true error exactly 0.5 on them. So stopping, which is what Novikoff guarantees, does not mean learning. The difference between the two rows is the number of mistakes per example. §4.4 bounds the expected true error by this ratio, and with coin flips the ratio is above 1, so the bound says nothing, as it should.

Second, what does guarantee learning. By §4.4, with i.i.d. data from a distribution with a margin, the expected true error is at most $(R/\gamma)^2/(n+1) = 200/(n+1)$, since $R^2 = 2$ and $\gamma = 0.10$. To test this I trained on 20 independent training sets for each n, all from one distribution, and measured the true error of each result on the same 200,000 new points (Figure 8).

notebook §4 Does stopping mean learning? · In [7]

```
# one fixed distribution (its w*) for every training set
ws = np.random.default_rng(21).normal(size=5)
ws /= np.linalg.norm(ws)
# 200,000 fresh points from it, never trained on, to measure the true error
Xt, yt, _ = make_separable(200_000, d=5, gamma=0.10, seed=999, w_star=ws)
# (R/gamma)^2 with R^2 = 1 + 1 (unit sphere + bias coordinate) and gamma = 0.10
BOUND = 2 / 0.10 ** 2

rows = []
for n in [30, 100, 300, 1000, 3000]:
    fits = []
    for s in range(1000, 1020):
        X, y, _ = make_separable(n, d=5, gamma=0.10, seed=s, w_star=ws)
        fits.append(Perceptron().fit(X, y))
    M = np.mean([f.total_mistakes for f in fits])
    err = np.mean([1 - accuracy(f.w_last, Xt, yt) for f in fits])
    rows.append({'n': n, 'mistakes': M, 'mistakes / (n+1)': M / (n + 1),
                 'bound (R/gamma)^2/(n+1)': BOUND / (n + 1), 'true error': err})
lc = pd.DataFrame(rows).set_index('n')
display(lc.round(5))
```

Figure 8. The experiment that tests the bound of §4.4.

Lines 2–3 fix one $w^\star$, so every training set comes from the same distribution. Line 5 draws the 200,000 evaluation points with a seed no training set uses. Line 7 is the top of the bound, $2/0.10^2 = 200$. I take it from how the data is built and do not measure it, because §4.4 needs the margin of the whole distribution, and line 13 of Figure 1 makes that 0.10. Lines 12–14 train on 20 training sets for each n. Line 15 averages the mistakes and line 16 averages the true errors, so they estimate the expected values in §4.4. Lines 17–18 store them with the mistakes per example and the bound for that n.

Averages over the 20 training sets:

n	30	100	300	1,000	3,000
mistakes	11.6	18.5	23.5	28.0	33.2
mistakes / $(n+1)$	0.374	0.183	0.078	0.028	0.011
bound $(R/\gamma)^2/(n+1)$	6.45	1.98	0.664	0.200	0.067
true error, measured	0.0468	0.0070	0.0021	0.00039	0.00001

(The second row divides the mistakes on n points by $n+1$, which is close to the $\mathbb{E}[M]$ on $n+1$ points in §4.4.)

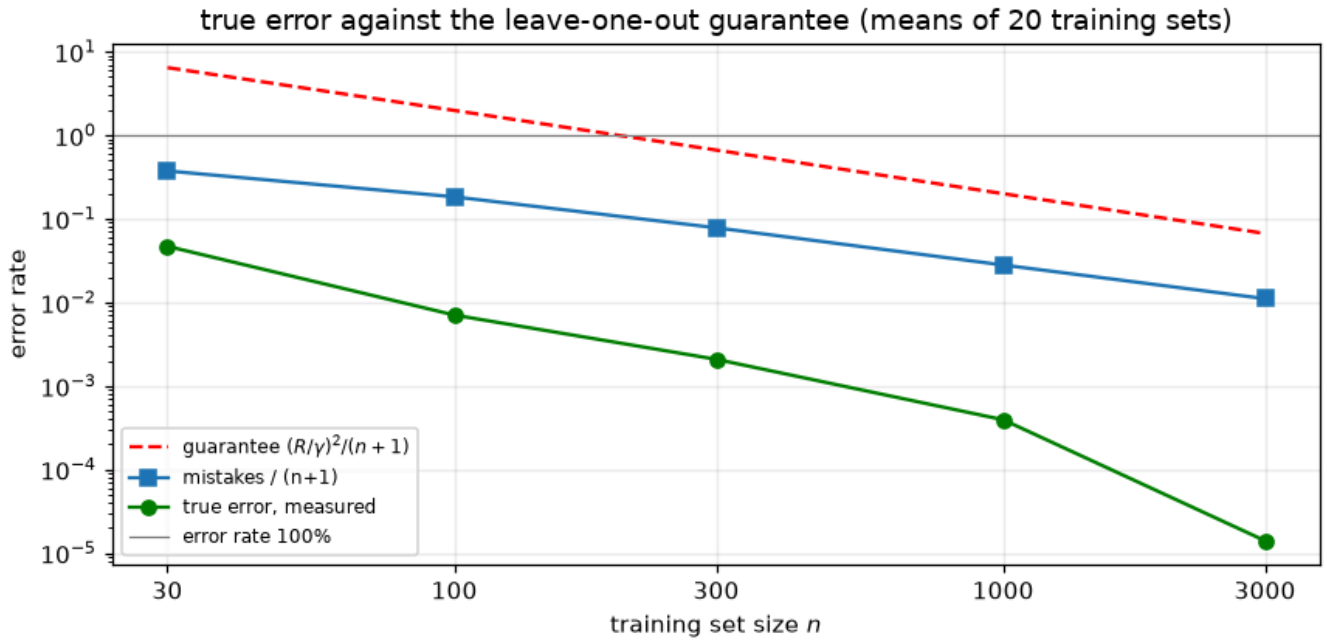


Figure 9. The same three rates on a log scale. At every n the guarantee (red, the bound of §4.4) is above the mistakes per example (blue), which is above the measured true error (green). Above the grey line the bound is more than 1 and says nothing.

What the table shows:

1. **It learns.** When n grew 100 times, the true error fell from 0.0468 to about 0.00001 (the smallest values are less accurate; see 4).
2. **The bound held at every** n, but it is very loose: more than 100 times the measured error, and 1 or more (useless) for n below 200.
3. **The mistake count went up, not down.** It roughly tripled, from 11.6 to 33.2, while the true error kept falling, and it can never go above 200. So the number the theorem controls is not the number that measures learning. Divided by $n + 1$, though, it bounds the expected true error, and at every n it lies between the bound and the measured error.
4. **Measurement accuracy.** A true error measured on 200,000 points is within ± 0.003 of the exact value with 95% confidence (Hoeffding, §7). The last three values are smaller than that, so Hoeffding only guarantees (with 95% confidence) that those errors are below about 0.005, still far below the bound.

So the answer is "not by itself". Every run in both tables stopped, and stopping alone says nothing about new data. What bounds learning is the number of mistakes per example, under i.i.d. sampling. A margin in the distribution keeps the mistake count below a fixed limit however large n is, and that is exactly why the true error falls.

6.3 With label noise the guarantee no longer applies

When some labels are flipped, no hyperplane separates the data. $w^\star$ still exists, but some points are on its wrong side, so there is no margin $\gamma > 0$ and inequality (a) cannot be used. The perceptron never stops, and it simply runs until the epoch limit. The bound of §4.4 also disappears, because it relied on $M \le (R/\gamma)^2$.

The bigger problem is what the textbook algorithm returns: whatever weight vector the last update left, which is just one point on a trajectory that keeps jumping around. On data where each label is flipped with probability 0.06, over 400 epochs:

	training accuracy
returned by the textbook algorithm (last epoch)	0.9250
best epoch (epoch 7)	0.9275
worst epoch	0.7725

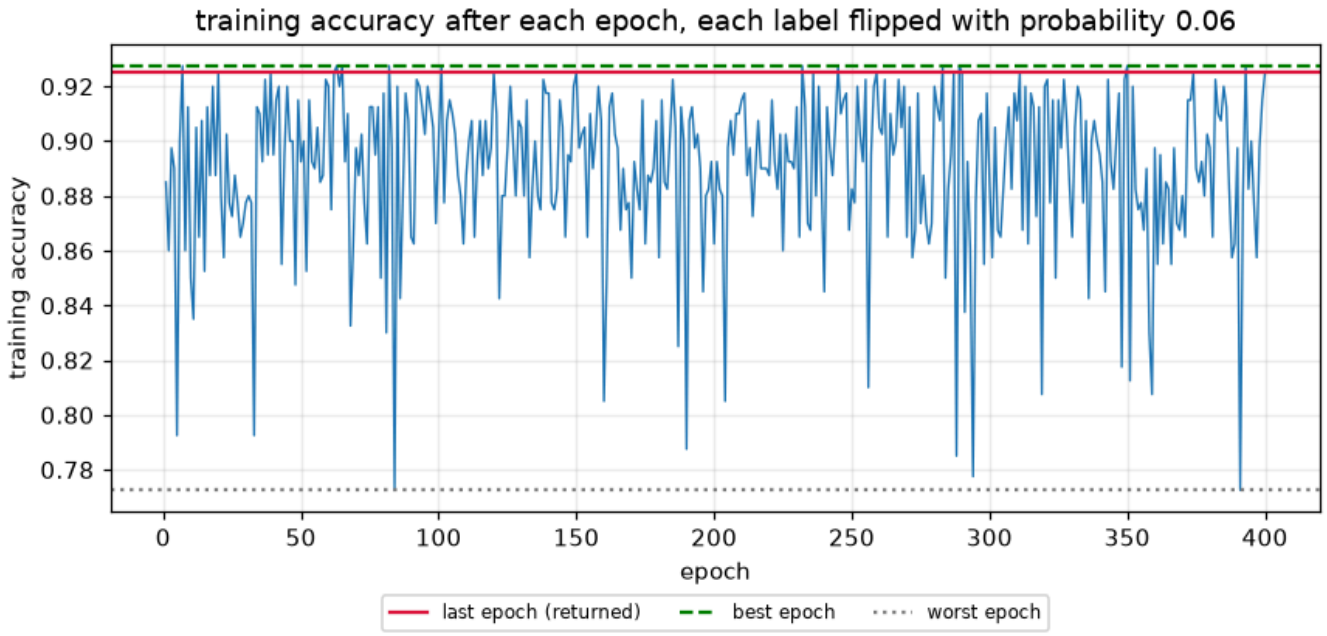


Figure 10. Training accuracy after each of 400 epochs, each label flipped with probability 0.06. It never settles, and the textbook algorithm returns wherever the last epoch ended (red).

The mistakes in the last five epochs were 64, 78, 76, 68 and 76, so training is not settling down. In this run the last epoch happened to be close to the best one, but nothing made that happen: the same run also went through 0.7725. Over 20 training sets (§8.2), the true accuracy of what the textbook algorithm returned ranged from 0.749 to 0.940. The algorithm has no idea of a "best so far" to keep.

7. Evaluation: three guarantees

With a margin and i.i.d. data the mistake bound leads to a bound on the expected true error (§4.4, §6.2). With noisy labels both disappear (§6.3). So I also need a guarantee that does not need a margin: one for an accuracy measured on held-out data.

This is Hoeffding's bound (module 3.9). For one fixed classifier tested on m independent points, with probability at least $1-\delta$,

$$\left|\text{measured error} - \text{true error}\right| \;\le\; \sqrt{\frac{\ln(2/\delta)}{2m}}$$

It needs no margin, only that the test points are i.i.d. and were not used to choose the classifier. Module 3.9 stresses that the bound is for a fixed hypothesis; if the hypothesis is chosen using the same data, a more advanced argument is needed. That is why I choose the version on validation data and use the test set only once, for the chosen model.

	Mistake bound (§4.2)	Leave-one-out bound (§4.4)	Hoeffding (3.9)
bounds	mistakes during training	expected true error of the returned w	gap between a measured error and the true error

	Mistake bound (§4.2)	Leave-one-out bound (§4.4)	Hoeffding (3.9)
needs	a margin in the training data	a margin in the whole distribution, and i.i.d. data	i.i.d. test data not used for choosing
as data grows	stays the same	falls like $1/n$	falls like $1/\sqrt{m}$ (m = test size)
with label noise	no longer applies	no longer applies	still applies

With $\delta = 0.05$, the 2,000-point test set gives ± 0.030, and the 200,000-point true-error measurements give ± 0.003. As a check, module 3.9 computes the same formula with $m = 1000$ and gets ± 0.043; doubling m divides this by $\sqrt{2}$, which gives ± 0.030.

notebook §6 Two repairs, and choosing between them · In [9]

```
14 # choose on validation only, then use the test set once
15 chosen = tab['val acc'].idxmax()
16 test = accuracy(cands[chosen], Xte, yte)
17 true = accuracy(cands[chosen], Xtrue, ytrue)
18 # Hoeffding (module 3.9), 95%, for one fixed classifier on m test points
19 h = np.sqrt(np.log(2 / 0.05) / (2 * len(yte)))
20 print(f'chosen on validation: {chosen}')
21 print(f'test accuracy {test:.4f} +/- {h:.4f} | true accuracy {true:.4f} | inside: {abs(test - true) <= h}')
```

Figure 11. Choosing on validation, then using the test set once with Hoeffding's interval.

Line 15 chooses the version with the best validation accuracy. Lines 16–17 measure the chosen model once on the test set and once on 200,000 new points. Line 19 is the formula above with $m = 2{,}000$ and $\delta = 0.05$, and line 21 checks whether the true accuracy is inside the interval.

A check that real data cannot give. The chosen model scored 0.9370 on the test set, so Hoeffding puts its true accuracy in $[0.907, 0.967]$. Measured on 200,000 new points it is 0.9401, which is inside. The interval is also wide: the test estimate was off by about 0.003 (at most 0.006, counting the ± 0.003 of the 200,000-point measurement), while the bound allowed 0.030. Like the other two bounds, it is safe but loose.

---

8. Two repairs, and choosing between them

8.1 Two repairs

§6.3 showed two separate problems, so I tried two fixes.

Problem	Fix	Cost
the returned weights are arbitrary	**pocket**: keep the weights with the best training accuracy seen at the end of an epoch, a simple version of Gallant's (1990) pocket algorithm	one accuracy check per epoch and one copy of w
the weights keep jumping around	**averaged**: return the average of the weight vector after every example (Freund & Schapire, 1999)	one running sum

Both fixes are inside the same loop as the textbook rule, so one training run gives all three answers: vanilla (the textbook last weights), pocket and averaged (Figure 12).

notebook §1 Model · In [3]

```
20          w_sum, steps = np.zeros(d), 0
21          best_w, best_acc = w.copy(), -1.0
26          for epoch in range(self.max_epochs):
27              errs = 0
28              # same order every epoch (the leave-one-out argument needs a fixed order)
29              for i in range(n):
37                  # averaged perceptron: running sum of w after every example
38                  w_sum += w
39                  steps += 1
40
41              acc = np.mean(np.where(Xa @ w >= 0, 1, -1) == y)
45              # pocket: keep the best w seen at the end of an epoch
46              if acc > best_acc:
47                  best_acc, best_w = acc, w.copy()
52          self.w_last = w
53          self.w_pocket = best_w
54          self.w_avg = w_sum / steps
```

Figure 12. Pocket and averaged, inside `Perceptron.fit`.

Line 20 starts the running sum and line 21 starts the pocket. Accuracy -1 makes sure the first epoch always replaces it. For averaging, lines 38–39 add the current w to the sum after every example, whether or not it was a mistake, and line 54 divides by the number of steps. For the pocket, line 41 measures training accuracy at the end of each epoch, and lines 46–47 keep a copy of w whenever it is better than the best so far. Lines 52–54 return all three; vanilla, the textbook answer, is just the last w.

8.2 Choosing on held-out data

The three versions come from one run on the 400-point sample of §6.3. Validation and test are new samples of 2,000 points each from the same distribution. I choose on validation accuracy and use the test set once, afterwards.

version	train acc	val acc	val perceptron loss	$\|w\|$
vanilla (last weights)	0.9250	0.9265	0.1273	4.01
pocket (best epoch), **chosen**	0.9275	0.9310	0.0851	2.65
averaged	0.9275	0.9270	0.1100	3.44

Pocket had the best validation accuracy. On the test set it scored 0.9370 ± 0.030 (Hoeffding, 95%), and its true accuracy on 200,000 new points is 0.9401.

Expected versus actual. With each label flipped with probability 0.06, the best any classifier can do is the accuracy of $w^\star$ itself, which is 0.9401 on the same 200,000 points. In this run the chosen model reached it. But one run is one draw of luck.

Twenty training sets. So I repeated the comparison on 20 new training sets of 400 points and scored every result on the same 200,000 points:

true accuracy	vanilla	pocket	averaged
mean	0.8591	0.9298	0.9381
standard deviation	0.0644	0.0209	0.0034
range	0.749 – 0.940	0.847 – 0.940	0.928 – 0.940

Averaging adds 7.9 percentage points of true accuracy on average (0.8591 to 0.9381) and removes most of the luck. Vanilla ranged from 0.749 to 0.940, while averaging was always within 0.012 of the ceiling and within 0.002 of it on average. Pocket, which validation chose in the single run above, was less reliable: usually close to the ceiling, but 0.847 in its worst run. I did not test whether this changes with a shuffled order. So the single validation choice was right for that one run, but the 20 runs are what support recommending averaging.

---

9. Discussion

9.1 Answer to the research question

Not by itself. What guarantees learning is a small number of mistakes per example, with i.i.d. data. A margin in the distribution itself is what keeps that number small.

What the theorem gives. It holds: both inequalities of the proof held at every mistake of the recorded run, which stopped after 18 mistakes when the bound was 193. It is loose because it covers the worst possible order.

Why that alone is not learning. The theorem is about a sequence of examples in any order. It says nothing about a distribution or about new points. With coin-flip labels the perceptron stopped with every training point correct and was no better than guessing on new points.

What turns it into learning. Two conditions are enough: training and future points come independently from the same distribution, and that distribution (not just the sample) has a margin. Then the leave-one-out argument gives $\mathbb{E}[\text{true error}] \le (R/\gamma)^2/(n+1)$. These conditions are sufficient, not necessary: without a margin the perceptron may still learn, but this bound can no longer show it (§9.5). What I first took as evidence against learning, that the bound does not grow with n (§10.2, challenge 4), is actually why it works: a number of mistakes that can never pass a fixed limit, spread over more and more examples, is a falling rate. Measured: as n grew 100 times, the true error fell from 0.0468 to about 0.00001 (the smallest values are less accurate, §6.2), always far below the bound.

When the guarantees disappear. With label noise there is no margin, so there is no mistake bound, no stopping and no leave-one-out bound. That is exactly where the textbook algorithm has no defined result: over 20 training sets, the true accuracy of what it returned ranged from 0.749 to 0.940. Averaging fixes this for the price of one running sum. It added 7.9 percentage points on average and came within 0.002 of the best possible accuracy. Hoeffding's bound still applies there, and its interval contained the true accuracy.

So the three guarantees do different jobs. Novikoff says whether and how soon training stops. The leave-one-out bound says what to expect of the result, given a margin and i.i.d. data. Hoeffding says how far a measured number can be from the truth, given only independent test data.

9.2 Loss function versus task objective

The ideal comparison. The desired output for x is the label $w^\star$ gives it, and the ideal way to compare the model's output with it is the 0/1 error over the whole distribution, the true error. Training cannot use it, because the 0/1 error is flat almost everywhere, so the perceptron uses L instead (§3.3). Evaluation does use it: estimated on validation and test data, and, only because the data is synthetic, measured directly on 200,000 new points. On a real dataset only the test estimate with Hoeffding's interval would be available. Training on a substitute means the loss and the accuracy can disagree, in three ways:

- **The loss depends on the length of w; the predictions do not.** $L(cw) = cL(w)$ for $c > 0$, but no prediction changes. This is also why the learning rate does not matter (§3.5). In §8.2 the losses differ mostly because the weight vectors have different lengths (2.65 to 4.01): divided by $\|w\|$, all three are within 0.0004 of each other. So I never use L to choose a model; I use validation accuracy.
- **The loss counts distance, the task counts errors.** L depends on how far wrong points are on the wrong side, not on how many there are. The extreme case is $w = 0$: $L = 0$, the minimum, but it classifies nothing (§3.3). This is why the algorithm's procedure, not the loss, does the work.
- **The loss cannot reach zero with noise.** With each label flipped with probability 0.06, even $w^\star$ has a true error of 0.06 and a positive loss, and every flipped point causes an update every time it is visited. So the algorithm keeps chasing errors that cannot be fixed, which is the jumping around in §6.3. The fixes are pocket and averaging, and measuring against the ceiling (0.9401) instead of 100%.

9.3 What this says about the perceptron

The update rule itself is not the problem. It has a guarantee, the guarantee holds in practice (§6.1–6.2), and with i.i.d. data from a distribution with a margin it is a real guarantee about learning. The problem is that the model leaves things out in two ways. The hypothesis space is too small for problems like XOR (Minsky and Papert's point); that cannot be fixed inside $\mathcal{H}$ and needed a different kind of model. And the algorithm says too little: it does not keep the best weights so far and has no defined result when the data cannot be separated, which is cheap to fix (§8). It also has no preference between the many separating hyperplanes it could find, which this project does not measure (§9.5, item 3).

9.4 Limitations

- $\mathcal{H}$ is a single hyperplane, so XOR cannot be represented.
- All data is synthetic, which is the only way to know $w^\star$ and γ and to measure the true error. So nothing here is evidence about a real dataset. The distribution is also an easy one: the band of width 2γ around the boundary is empty, so any w close enough to $w^\star$ makes no errors at all. This is part of why the true error in §6.2 falls so fast.
- The leave-one-out bound is about the average over training sets, not one particular training set, and it needs a fixed order (§4.4).
- The smallest true errors in §6.2 are below the ± 0.003 accuracy of the measurement.
- The true error in the coin-flip check ($d = 500$) is measured on 5,000 points instead of 200,000, because 200,000 points in 500 dimensions would be slow.

9.5 Future work

1. **Remove the empty band.** Repeating §6.2 on a distribution with points arbitrarily close to the boundary would show how much of the fast learning comes from the band. The bound would say nothing there, so only the measured true error would be left.
2. **Check the voted perceptron's own bound.** Freund and Schapire prove their bound for the voted perceptron. I recommend averaging, which has no bound in this report (§10.4). Implementing voting and checking its bound against the measured true error would show how much guarantee averaging gives up.
3. **Compare with the maximum-margin separator.** The bound gets better with a larger margin, but the perceptron does not look for a large margin. Measuring how far its solution is from the maximum-margin one would put a number on the last point of §9.3.

---

10. Implementation log

10.1 What is mine

Component	Source
`Perceptron`: update rule, pocket and averaged versions, recording the proof's quantities	mine, NumPy
`perceptron_loss`, `predict`, `accuracy`, `bound_inputs`	mine
`make_separable`: data with a known margin, and more data from the same distribution	mine
`draw`: noisy samples for training, validation, test and the true-error measurement	mine

No machine-learning library is used: NumPy for computation, pandas for tables and matplotlib for plots.

10.2 Challenges and solutions

1. **Choosing what to study.** I first considered other topics that applied existing models to real data. While working on them I realised that my knowledge of the models themselves was not deep enough, so I decided to use this assignment to study one model in depth instead. I chose the perceptron, because it is simple enough to implement by hand and its guarantee can be tested directly.
2. **Where the bias goes.** Novikoff's theorem assumes a boundary through the origin, but a classifier needs a bias. Adding a constant 1 to every input solves this, but changes the radius to $R^2 = 2$. A bound computed with the old R would be for a different problem, so `bound_inputs` measures R and γ after adding the 1.
3. **Controlling the margin.** Random points can have an arbitrarily small margin, so the bound could not be set in advance. Throwing away points closer than γ to the boundary fixes the margin, and putting points on the unit sphere fixes $R = 1$ before the bias coordinate is added. Later the generator had to accept an existing $w^\star$, so that new points could come from the same distribution. That is what made the true error measurable.
4. **My first answer to the research question was wrong.** An earlier version answered "no", because Hoeffding's bound shrinks with n while Novikoff's does not. That compared a total count of training mistakes with the accuracy of a test estimate, and missed that a total with a fixed limit, spread over more examples, is a falling rate. Measuring the true error of the returned w showed it falling steadily with n, which the old answer could not explain and the leave-one-out argument predicts. I added §4.4 and §6.2. A later check with coin-flip labels showed that i.i.d. data alone is not enough: the perceptron stopped and learned nothing. The margin has to belong to the distribution, not just to the sample. That check is now the first part of §6.2.
5. **Removing what the question did not need, and making the settings consistent.** Earlier versions had a margin variant of the algorithm, sweeps over the margin, n, d and the learning rate, and separate unit tests. None of them helped answer the question, so I removed them; the proof recording doubles as the check of the code (§5), and the learning-rate result is kept as a short derivation (§3.5). The noisy experiments also used different settings from the rest (margin 0.12 instead of 0.10, exactly 6% of training labels flipped, shuffled order) for no reason. Making them consistent changed two claims that had rested on a single run, so claims about the repairs now rest on 20 training sets.
6. **Choosing on the test set.** An early draft ranked the versions by test accuracy, which means the test set was used for choosing. Now §8.2 chooses on validation and uses the test set once.

10.3 Use of AI tools

With the help of an AI assistant (Claude), I chose the topic (it suggested Novikoff's theorem, §2.2), wrote the code and this report, and made the revisions along the way. I discussed each step closely with it and reviewed its suggestions critically instead of accepting them: every claim was checked against a measurement or a derivation, and changed when it failed. Examples of AI-drafted material that failed this check:

- the first answer to the research question was wrong (§10.2, challenge 4);
- the experiments used different settings for no reason, and some claims rested on a single run (challenge 5);
- an early draft chose between the versions on the test set (challenge 6);
- drafts of this report contradicted themselves in places, for example saying that the number of mistakes does not grow with n, while the table in §6.2 shows it growing.

The conclusions do not rest on the assistant: the proof (§4.2, §6.1), the leave-one-out bound (§6.2), the code (§5) and every number in this report are checked by the notebook. The parts I do not fully understand are listed in §10.4.

10.4 Knowledge gaps

These are components I use but do not fully understand. For each: why it is needed, how I checked it, and what I did to understand it.

- **The averaged perceptron's guarantee.** I use averaging, but the leave-one-out bound of §4.4 only covers the last weight vector. Freund and Schapire prove their bound for the voted perceptron, and I do not derive one for averaging. *Why it is needed:* averaging is the version I recommend for noisy data (§8.2). *How I checked it:* over 20 training sets its true accuracy, measured on 200,000 points, was always within 0.012 of the best possible and within 0.002 on average. *What I did to understand it:* I compared averaging with voting. Voting weights each weight vector by how many examples it survived, and my running sum (line 38 of Figure 12) adds w after every example, which weights each weight vector by how many steps it was kept, almost the same weights as voting. But averaging adds up the weight vectors, while voting adds up their predictions, and I could not turn this into a bound for averaging myself.
- **Hoeffding's inequality.** I use the result from module 3.9 but have not worked through its proof. *Why it is needed:* with noisy labels it is the only guarantee left, and it tells me how far a measured accuracy can be from the true one (§7). *How I checked it:* my arithmetic matches the worked example on page 3.9 ($m = 1000$ gives ± 0.043, and doubling m gives ± 0.030), and because the true accuracy is known here, I checked that the test interval contains it (0.9370 ± 0.030 against 0.9401). *What I did to understand it:* I followed its conditions in module 3.9 (one fixed hypothesis, independent test points). That is why I choose on validation data and use the test set only once. The module says that a hypothesis chosen on the data needs a union bound and VC theory, which I have not studied, so I put no interval on the validation comparison.

References

Freund, Y., & Schapire, R. E. (1999). Large margin classification using the perceptron algorithm. *Machine Learning*, 37(3), 277–296.

Gallant, S. I. (1990). Perceptron-based learning algorithms. *IEEE Transactions on Neural Networks*, 1(2), 179–191.

Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Novikoff, A. B. J. (1962). On convergence proofs on perceptrons. *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615–622.

Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.